

Elevation of Autonomy — Companion Guide

Session facilitation notes for the card-based threat modelling deck

Version 0.1

Who this guide is for

You're about to run a threat modelling session using *Elevation of Autonomy*. Maybe it's your team's first card-based session, maybe you're an experienced EoP facilitator extending into AI systems, maybe you've been handed the deck and told to "just run it." This guide is for all three of you.

By the end of this document, you should be able to:

- Plan and run a 90-minute session from cold
- Recognise the red flags that mean a session is drifting
- Ask the five questions that rescue a session that's lost direction
- Produce a post-session artefact that lives in the repository and actually gets read

The full rules live in the *Rules* document. The cards themselves live in the *Print and Play* document. This guide is about what happens in the room.

Before the session

Pick the right system to model

Elevation of Autonomy is built for AI, LLM and agentic systems. If the system in scope has none of the following, play EoP or Elevation of Privacy instead:

- A language model in the request path
- Retrieval-augmented generation (RAG) or long-term memory
- Tool-calling, function-calling, or MCP integrations
- Multi-agent orchestration
- Fine-tuned or prompt-tuned behaviour that ships with the product

If the system has **some** of those features but is mostly a traditional application with an LLM bolted on the side, you can play Elevation of Autonomy alongside EoP — deal EoP first, Elevation of Autonomy second, or mix the decks if your players have played both before.

Produce the data flow diagram first

Everything in the session hangs off a shared DFD. No DFD, no session. If the team hasn't drawn one, the first thirty minutes of the session becomes a DFD workshop — which is fine, but change the calendar invite to reflect it. Do not try to draw the DFD and play the cards in the same hour.

The DFD should show:

- Every external actor (users, other services, attackers)
- Every trust boundary (authentication zones, tenancy boundaries, customer vs internal)
- Every data store (databases, vector stores, memory stores, caches, logs)
- Every external dependency that sits in the request path (models, APIs, MCP servers)

If the DFD contains a box labelled "the agent" with no internal detail, it is not a DFD; it is a placeholder. Open the box.

Nominate a scribe before the first card is played

The scribe's job is to capture every accepted threat in the register before the next trick is led. It is not a passive role — a good scribe will push back on vague threat descriptions ("can you be more specific about which tool?") and force the table to commit to a mitigation before moving on. Rotating scribes across tricks is fine; leaving the role unassigned is not.

Set the scope explicitly

"We're threat modelling DevAssist" is too broad. "We're threat modelling the retrieval path and the `create_pr` tool in DevAssist v2.3" is a scope. Write the scope at the top of the whiteboard. Any card application that falls outside the scope is recorded as "out of scope, raise in next session" and passed over.

The 90-minute session

0–15 minutes: Frame and warm up

- Walk the DFD. Confirm trust boundaries. Confirm the scope statement.
- Read the short-form rules aloud. Yes, even to EoP veterans.
- Deal the cards. Announce the lead player.
- Ask the five questions (see below) as a warm-up. The answers often surface the first two or three threats before any card is played.

15–75 minutes: Play

- Cards are played, threats are described, the scribe captures each one.
- The facilitator watches for the red flags below and intervenes when they appear.
- If a mitigation discussion runs past 90 seconds, record "open" and move on. Long mitigation debates are for the follow-up, not the session.

75–90 minutes: Wrap-up

- Review the threat register. Agree owners for every open mitigation.

- Agree a date for the follow-up — typically one to two weeks out — and put it in the calendar before the team leaves the room.
- Sign off. Every player adds their name to the artefact as a participant.

The five questions to ask every session

If you only have time to ask five questions, these are the five. Ask them as a warm-up at the start of the session, and ask them again at the wrap-up to confirm nothing was missed.

1. **What reaches the model's context, and from where?** Every input — prompts, retrieved documents, tool outputs, memory reads, peer-agent messages — is a potential injection vector. If you cannot enumerate the sources, you cannot defend them.
2. **What can the agent actually do?** Name every tool, every scope, every credential the agent holds. For each one, name the blast radius on compromise. If the answer is "it depends," you have a finding.
3. **Which decisions does the model make that matter?** Any consequential decision — authorisation, routing, approval, filtering — that depends on model output is a finding in waiting. Move the enforcement into a deterministic layer or accept that the decision is probabilistic.
4. **What personal data does the system touch, and what does it generate?** The T.R.I.M. question. Transfer, Retention, Inference, Minimisation. If the system generates personal data at any step, Articles 5, 16, and 17 of the GDPR apply — and most designs don't have an answer.
5. **What happens when a dependency changes silently?** Model versions, prompt templates, tool descriptions, MCP server behaviour, memory state. If any of these can change without the team noticing, the invisible-dependency card is in play (Trump II).

These five questions cover roughly 70% of the findings any session will produce. The cards exist to surface the other 30% and to force the team to be specific about each one.

Red flags during the session

Watch for these. They mean the session is drifting and the facilitator needs to intervene.

Red flag: the same component gets every card played against it

If three consecutive tricks all target "the retrieval layer," either the retrieval layer is genuinely the riskiest thing on the DFD (in which case, great, keep going) or the rest of the DFD is invisible to the team and needs explicit attention. Call "Spotlight" on a neglected component.

Red flag: threats are getting generic

"An attacker could prompt-inject the model." Specific to what? Via which channel? What do they get? Generic threats are the scribe's problem to flag. If the scribe hasn't flagged it, the facilitator should.

Red flag: the architect is dismissing everything

"We have a WAF." "The model won't do that." "Our training data prevents this." These are not mitigations. A genuine mitigation names a specific control, a specific location in the architecture, and the specific failure mode it prevents. If the architect cannot name all three, the threat stays open.

Red flag: Trumps are being hoarded

Non-adversarial hazards are the hardest to spot and the most valuable to surface. If a player has three Trumps at turn 20 of 27 and hasn't played any of them, the facilitator should gently remind the table that Trumps played late are worth less to the design than Trumps played early. Consider invoking the Trump Cooldown rule in a future session.

Red flag: silence

If a player hasn't contributed for three consecutive tricks, the facilitator should deliberately bring them in. "Dana, you've been quiet — what card would you play on this component?" Silence in a threat modelling session usually means either disengagement or that the player has a concern but doesn't want to voice it. Either way, surfacing it is the facilitator's job.

Red flag: no-one has played a Clubs card

If personal data is anywhere in the system and the Clubs suit hasn't appeared by the mid-session mark, invoke the Privacy Round (see full rules). Privacy threats are routinely under-played because they feel less "technical" than adversarial threats. They aren't.

Red flag: mitigations keep defaulting to "add a prompt filter"

Prompt filters, blocklists, and refusal-trained models are all defences against a vanishing fraction of the adversarial subspace. If three mitigations in a row are variants of "we'll filter for this," play Trump VII — Adversarial Subspace — yourself, and force the discussion back to structural mitigations outside the model.

The post-session artefact

A session that produces no artefact did not happen. The artefact is the output, the cards are just scaffolding.

The post-session template should capture:

- **Scope statement.** What was in scope, what was explicitly out.
- **DFD snapshot.** Either embedded inline (Mermaid works well) or linked to the canonical version.
- **Threat register.** One row per accepted threat. Columns: card ID, affected component or flow, specific example described, proposed mitigation, owner, status.
- **Accepted risks.** Threats the team chose not to mitigate, with the business rationale.
- **Open items.** Threats recorded with "mitigation: open" — these are the facilitator's follow-up list.
- **Participants.** Every player, by name, with the date.
- **Sign-off.** At minimum: tech lead, security representative. Depending on the system: product owner, architect, privacy representative.

The template should live in the repository alongside the code it describes, in a file like `THREAT_MODEL.md` or `docs/threat-model.md`. It should be version-controlled, updated when the design changes, and reviewed as part of any significant feature work that touches the AI layer.

Note to engineering leaders: a threat model that lives in a Confluence page nobody updates is worse than no threat model at all — it gives the illusion of rigour without the substance. The git-tracked markdown version is not harder to maintain; it is easier, because it travels with the code.

What to do between sessions

Threat modelling is a discipline, not an event. Between sessions:

- **Record which cards produced good threats and which didn't.** After three to five sessions, some cards will stand out as over- or under-powered. Prune the deck accordingly and submit feedback upstream.
- **Build custom cards for your system.** If your environment has recurring characteristics — multiple MCP servers, a particular multi-agent orchestration pattern, specific regulatory constraints — draft local cards that capture the threats that matter most to you. The deck is intentionally small (27 cards) because the AI threat surface is still being mapped; treat it as a starting point, not a finished taxonomy.
- **Revisit the threat register.** Open items should have owners and target dates. If an item is still open three months later with no movement, it needs escalation or a documented accepted-risk decision.
- **Run a retrospective after every third or fourth session.** What did the cards surface that you wouldn't have spotted otherwise? What did they miss? Feed the answers back into card selection and facilitation technique.

Running a session on something other than DevAssist

The reference examples in my writing all use DevAssist, an internal developer productivity agent. Your system is different. A few adaptations worth knowing.

If the system is a customer-facing chatbot

- Focus heavily on the Clubs suit — personal data regulation is your dominant concern.
- Pay particular attention to ♥A (Excessive Agency) and ♠A (Prompt Injection) — customer-facing systems have the widest input surface.
- Trump V (Context Rot) applies strongly to long customer conversations.

If the system is an internal code or document assistant

- The DevAssist playbook applies directly.
- ♠Q (Supply Chain Compromise) is especially relevant because internal assistants tend to sit on top of sprawling dependency trees.
- Trump II (Invisible Dependency) is the one to watch — prompt templates, tool descriptions, and model versions are rarely in anyone's SBOM.

If the system is a multi-agent system

- Hearts suit dominates. Expect ♥Q (Inter-Agent Trust) and ♥J (Cascading Failure) to come up repeatedly.
- Add extra time for the DFD — multi-agent systems produce complex graphs that take longer to walk.
- Consider running two sessions: one on the agent-to-agent boundary, one on the agent-to-user boundary.

If the system is a high-stakes classifier (fraud, moderation, medical)

- Trumps dominate. Trump VI (Geometric Attack) and Trump VII (Adversarial Subspace) are the foundational findings. Almost every adversarial threat in Spades reduces to one of these.
- The key question is whether any consequential decision is being made by the model alone, or whether a deterministic enforcement layer exists downstream. If the former, the model is carrying a guarantee it cannot carry.

When you're ready to run your first session

1. Block 90 minutes on the calendar.
2. Invite the right players — architect, tech lead, security representative, a privacy voice if personal data is in scope, one or two engineers who actually build the thing. Six is the ceiling.
3. Draw the DFD, or schedule a prep meeting to draw it.
4. Print the deck, or open it in your Kanban tool.
5. Open the post-session template. Name a scribe.
6. Deal. Play. Capture.

The first session will feel clunky. The second will feel natural. By the third, the team will be asking when the next one is scheduled.

Feedback

This is version 0.1. The next revision of this guide will incorporate what I've learned from running and observing at least five sessions across different system types. If you run a session and something in this guide was wrong, missing, or unhelpful, tell me — the next version will be better for it.

Happy threat modelling.

Stay tuned!

Tags: threat-modelling, elevation-of-autonomy, AI-security, facilitation, runbook